# Adventures in Reproducing and Integrating Open-Source: Developing Guidelines for Modular Software Components in Robot Grasping and Manipulation Pipelines for Picking in Clutter

**Huajing Zhao**[1], **Brian Flynn**[1], **Adam Norton**[1*], **Holly Yanco**[2]
[1]University of Massachusetts Lowell, Lowell, MA, USA;
[2]University of Massachusetts Amherst, Amherst, MA, USA
[*]Corresponding author: adam_norton@uml.edu

## Abstract

When developing a novel software component of a robot system, researchers typically integrate open-source code for other components in the pipeline, such as perception, grasp planning, and motion planning. However, there are several bottlenecks related to the accessibility and reproducibility of open-source which are stifling research, development, evaluation, and deployment of robots. To this end, we are working to develop community-driven guidelines for modular components in robot software pipelines in the context of picking in clutter. In this work, we investigate reproducing and integrating multiple perception and grasp planning components used in vision-based perception-to-action pipelines. The architecture and methodology used to conduct these exercises are reviewed, highlighting issues encountered along the way. Recommendations for guidelines to improve accessibility, reproducibility, and integration of modular open-source components are shared related to the assets included in an open-source release, methods to modularize components, and practices for development and testing.

## Introduction

Executing grasping and manipulation tasks with a robot system requires the integration of many hardware and software subsystems. Most researchers generally contribute one novel component at a time and the other subsystems used are generally not standardized. This means that in order to develop and deploy the novel contribution, the remaining components in the pipeline must be available for a researcher to easily integrate (i.e., the components should be modular). While the advent of open-source contributions in robotics research should support this issue (i.e., leverage open-source components for the parts of the pipeline not being developed), there are many roadblocks to this process. Middleware like the Robot Operating System (ROS) (Quigley et al. 2009) has provided a common software platform for robotics research, but these issues still persist at both the core open-source code level (e.g., algorithms) and the ROS package level. These issues contribute to bottlenecks in the current robotics ecosystem, impeding many facets of research, development, evaluation, and deployment.

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

Beyond broader issues with a lack of software documentation (e.g., (Aghajani et al. 2019; Pinho et al. 2024)), the intertwined nature of robotics components and their dependencies have led to more specific issues with robotics code reproducibility being studied (e.g., (Cervera 2018; Lynnerup et al. 2020; Cervera 2023)). There is also a lack of standardization in the development and deployment of open-source code, such as how robot descriptions (e.g., URDF) are defined (Tola and Corke 2023, 2024). Other issues with misconfigurations in ROS (Canelas et al. 2024) as well as a lack of distribution in leadership of core package development and maintenance (Kolak et al. 2020) has further contributed to fractures in the community. Open-source in robotics should improve accessibility and reproducibility of functionality, thereby validating the results of research contributions and enabling performance comparison. Typical recommendations are to utilize container tools like Docker[1] so code can be executed within isolated environments to avoid issues with dependencies (Cervera 2018, 2025).

As part of the COMPARE Ecosystem[2], we are working to establish community-driven guidelines for improving accessibility, reproducibility, and integration of modular open-source software components in robot grasping and manipulation pipelines. In the context of vision-based perception-to-action pipelines for picking in clutter, we present the architecture and methodology of this endeavor by defining components and pipelines, proposing methods to improve modularity, conducting exercises to reproduce, modularize, and integrate components into pipelines, and deploying them to test their functionality. The results of our adventures in reproducing and integrating open-source are reviewed toward deriving recommendations for guidelines.

## Scope

Our initial use case is picking in clutter via vision-based perception-to-action pipelines using a single robot arm and a parallel jaw gripper, for which there are many types of open-source products available. While other use cases in robot manipulation will also benefit from these efforts (e.g., assembly, cloth manipulation), the capabilities for picking in clutter are often leveraged in these applications, making for

[1]https://www.docker.com/
[2]https://www.robot-manipulation.org/

a good starting point. These capabilities are driven by a set of components working together in a pipeline, for which most research will leverage multiple open-source components to work with the novel component(s) contributed (e.g., OVAL-Grasp (Tong et al. 2025) integrates Contact-GraspNet[3] for grasp planning and PartGLEE[4] for scene segmentation, Um et al. (Um et al. 2026) uses YOLOv8[5] for object detection).

Modularity of these components in pipelines not only improves accessibility and reproducibility, but also enables performance comparisons. Conducting benchmarking exercises to evaluate performance is becoming increasingly common. For example, GRASPA (Bottarel et al. 2023) compares three grasp planners under a common software framework, SceneReplica (Khargonkar et al. 2024) compares multiple grasp planners and perception methods for "model-based" and "model-free" grasping, and Rameshbabu et al. (Rameshbabu et al. 2025) compare four vision-based grasping algorithms. These works essentially produce evaluations of both the combinations of components in the pipelines and the components themselves by executing holistic pipeline benchmarking.

We consider open-source software at two levels: *components* that drive the underlying functions of a manipulation capability (e.g., scene segmentation, pose estimation, grasp planning, motion planning, control) and *pipelines* that integrate multiple components together to perform a task (e.g., pick and place). The guidelines we are building will cover the use of components and pipelines for both the execution of robot capabilities (e.g., object grasping) and benchmarking procedures (e.g., lifting and shaking the object to demonstrate a firm grasp), such that components can be easily integrated and swapped in and out (i.e., making them modular and interoperable). The goal is to lower the barrier to integration of open-source software, which in turn will improve reproducibility of capabilities between labs.

The reproducibility and integration exercises conducted in this paper are not intended to thoroughly evaluate the components and pipelines used. Rather, they are conducted to investigate and demonstrate best practices and effectiveness of modularity. Our goal was to produce pipelines that were functional (in order to explore solutions to the issues outlined), as opposed to rigorous evaluations of performance compared to their original publications.

## Components and Pipelines Methodology

This section reviews the salient aspects of our methodology to modularize components and integrate them into orchestrated pipelines. Additional documentation on our methodology for developing modular ROS 2-wrapped components and integrating them into pipelines is available online[6].

### Component Properties

While there are many types of open-source components that could be used when picking in clutter, we identify five primary categories based on the prevalence of research contributions made for each: perception, object selection, grasp planning, motion planning, and control. This breakdown also matches those defined and varied in similar works like GRASPA (3 perception methods, no object selection methods as only single object scenes were evaluated, 3 grasp planning methods, 1 motion planning method) (Bottarel et al. 2023) and SceneReplica (7 perception methods, 3 object selection methods referred to as "ordering", 6 grasp planning methods, 1 motion planning method) (Khargonkar et al. 2024). While control is a vital component that may be varied, we standardize using MoveIt[7] given its prevalence in the field and therefore do not vary it in the exercises conducted for this paper.

Across all components, properties like the language they are written in (e.g., C++, Python) and the compatibility of underlying libraries (e.g., Ubuntu 24.04, 22.04, etc.) are defined. The following properties are salient for each category with respect to their impact on hardware deployment (e.g., sensor data available for a given robot system) and integration requirements within a pipeline (e.g., perception output must match grasp planner input). Actively updated repositories of open-source components for picking in clutter, organized according to the specified characterization methods, are available online[8].

**Perception**. Several types of perception capabilities may be used, including object detection, segmentation, and pose estimation. All perception components will be trained on real and/or simulated data captured by one or more sensors in a variety of formats, including RGB images, depth images, depth images with RGB data (RGBD), or point clouds, among others. The format of the perception component output – depending on type – may be a filtered version of its input data (e.g., filtered point cloud, masked 2D image), a labeled set of data (e.g., identified objects in a scene), etc.

**Object Selection**. With multi-object scenes, the object to be grasped must be selected either automatically by the system or manually selected by the user. The former requires certain perception outputs; for example, following a prescribed object order (as in the Cluttered Environment Picking Benchmark (D'Avella et al. 2023)) assumes object labels are available, or a rule like picking closer objects first, which assumes object distances can be determined.

**Grasp Planning**. Grasp planners can use many types of planning, including analytical, direct regression, and sampling methods; see (Newbury et al. 2023) for a more thorough review. The data used to train grasp planners is generated from real and/or simulated experimentation, performed using one or more types of robot systems with variable end-effectors (e.g., suction, parallel jaw, multi-finger) and sensors (e.g., cameras, tactile; we do not consider the latter for this paper). Depending on the cameras available – and the perception components used in the pipeline – input data can vary: RGB images, depth images, point clouds, object masks etc. The number and position (e.g., top-down, variable, oblique) of the camera(s) will impact which grasp plan-

[3]https://github.com/NVlabs/contact_graspnet

[4]https://github.com/ProvenceStar/PartGLEE

[5]https://roboflow.com/model/yolov8

[6]https://flynnbm.github.io/ros2_methodology/

[7]https://moveit.picknik.ai/

[8]https://www.robot-manipulation.org/software

ner is deployed as well as its performance. The kind of object configurations the grasp planner is trained to work on (e.g., singulated, stacked, packed) will also impact how and when it is used. Lastly, its output type (e.g., 2D grasp rectangle, 6-DoF grasp pose, dexterous grasp) will impact the rest of the pipeline (i.e., how the target grasp will be used by the motion planner to plan a trajectory).

**Motion Planning**. For motion planning, we are primarily concerned with the class of planning algorithm being used (e.g., optimization-based, sampling-based) and how task-space goals are mapped to joint configurations, including whether inverse kinematics is required. Other useful properties for our use case are largely performance-based (e.g., time, path length, trajectory smoothness, success, and obstacle-avoidance performance on a set of benchmarking problems (Liu, Yap, and Khairuddin 2024)), which are instance-specific and therefore not considered for this work.

## Component Modularity

The open-source packages that comprise these components are typically made available through GitHub repositories, ideally with sufficient documentation covering the required dependencies and compatibilities (Pinho et al. 2024). To consider the components "modular," though, implies they can be easily integrated and swapped out for other components, requiring a certain architecture or other design paradigm be followed. In other words, a component is only as modular as the other components made modular following the same scheme. To this end, we standardize making open-source components modular by leveraging the ROS 2 ecosystem. The two prior works referenced – GRASPA (Bottarel et al. 2023) and SceneReplica (Khargonkar et al. 2024) – include open-source components wrapped in ROS 1; we advance this approach to using ROS 2 Jazzy (Ubuntu 24.04) and define four methods of making components modular, in order of preference:

**Method 1: Using a ROS 2 wrapper** if the underlying library is compatible with 24.04.

**Method 2: Using a ROS 2 wrapper + subprocess bridge** if the underlying library is not compatible with 24.04.

**Method 3: Using a ros1_bridge**[9] if a ROS 1 wrapper already exists and a dedicated machine to run it is available.

**Method 4: Updating the underlying library** to be compatible with 24.04 then using a ROS 2 wrapper (Method 1).

Wrapping functionality in ROS 2 requires existing or compatible message types, a SRV file (i.e., service-message definition), and a service server or node which wraps the underlying library for the component. We typically follow this high-level workflow when wrapping a library:

1. Identify the smallest useful domain API for the library.
2. Decide how to map the needs of the API map to ROS 2 interfaces: topics (e.g., streams and events), services (e.g., short request/response work), or actions (e.g., when a goal needs feedback, cancellation, or a result after longer-running work).
3. Define conversions from domain types to ROS messages.
4. Keep lifecycle, threading, and error translation explicit.
5. Add launch, configuration, and integration tests.

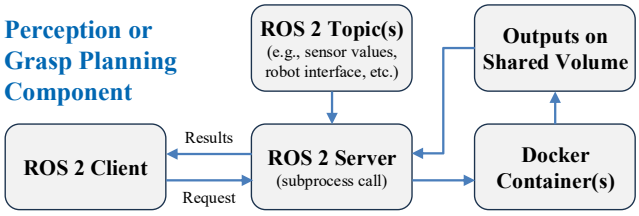



Figure 1: ROS 2 wrapper + subprocess bridge (Method 2) architecture for a perception or grasp planner component.

The subprocess bridge wrapping strategy (Method 2; see Figure 1) decouples ROS 2 communication from algorithm execution, thereby enabling the integration of models with incompatible dependencies into modern ROS 2 pipelines without modification of their original codebase. Using this method involves developing a lightweight ROS 2 node to provide a service that, upon receiving a request from a client, launches a Python subprocess[10] to scripts inside a Docker container. The output (e.g., segmentation results, grasp poses) is returned as a JSON or npy file that is parsed by the ROS 2 server and returned to the client as a SRV response. In this architecture, a server node is implemented to invoke the underlying algorithms through *subprocess.run()*, which executes `docker exec` to enter a Docker container configured with the legacy runtime environment required by the original codebase. The component's original Python script is executed inside the container while remaining isolated from the ROS 2 host system.

## Pipeline Orchestration

Orchestration methods for integrating ROS 2 components into an experimental pipeline for testing or benchmarking range from script-driven pipelines (e.g., sequentially executing nodes or launch files with arguments) to higher-level task execution control frameworks such as state machines and behavior trees. In the ROS ecosystem, examples include FlexBE[11], an open-source graphical front-end built upon the open-source SMACH state machine library[12], and MoveIt Pro[13], a proprietary graphical front-end with a developing ecosystem built upon the open-source BehaviorTree.CPP[14] behavior tree library. Other popular approaches include graph-based environments such as NVIDIA OmniGraph[15] within Isaac Sim, and block-based environments such as MathWorks Simulink[16]. While script-driven orchestration is functional and straightforward to implement, this method

[9] https://github.com/ros2/ros1_bridge

[10] https://docs.python.org/3/library/subprocess.html

[11] https://flexbe.readthedocs.io/

[12] https://wiki.ros.org/smach

[13] https://docs.picknik.ai/

[14] https://github.com/BehaviorTree/BehaviorTree.CPP

[15] https://docs.isaacsim.omniverse.nvidia.com/5.1.0/omnigraph/

[16] https://www.mathworks.com/help/ros/

| Type | Component | Input | Camera | Output | Modularity |
|---|---|---|---|---|---|
| Perception | PCL | Point cloud | Variable | Filtered point cloud | Method 1 |
| | UOC | RGB or RGBD image | Variable | Segmented RGB or RGBD | Method 2 |
| Grasp Planning | CGN | RGBD image | Oblique | 6-DoF grasp pose | Method 2 |
| | GPD | Point cloud | Variable | 6-DoF grasp pose | Method 1 |
| | GSAM* | RGB image | Top-down | 2D grasp rectangle | Method 2 |

Table 1: Component characteristics and modularity method. * Only the grasp detection functionality of GSAM was used.

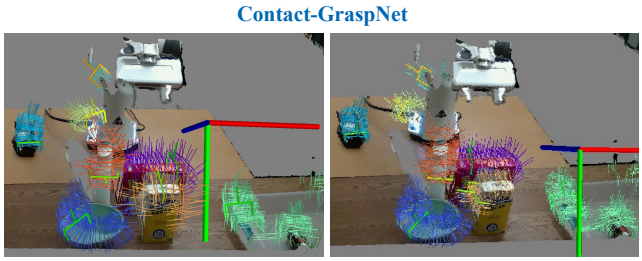



Figure 2: Comparison of the original CGN presented in its repository[3] and our reproduction on an example scene also from its repository.

tends to manage control implicitly and statically, limiting modularity, reuse, and runtime adaptability. Execution control frameworks, such as state machines and behavior trees, explicitly make control flow a primary focus, enabling conditional outcomes, recovery strategies, parallel execution, and structured reuse of sub-tasks. The ability to dynamically and quickly modify transitions, swap components, or adjust parameters per run without rewriting the pipeline or its components significantly improves experimental flexibility.

The choice of pipeline orchestration method should not determine whether a component can participate in the pipeline. Tightly coupling a utility to one orchestration tool limits where it can be reused, potentially forcing users to adopt unfamiliar tooling before they can benefit from the underlying capability. The use of ROS 2 interfaces to modularize components creates a boundary between these choices, enabling users to integrate them in the orchestration method of their choosing without requiring a different implementation for each environment. Accordingly, when building a ROS 2 service or action, the requests and responses should use existing ROS 2 message types. If a custom message type were used instead, then the conversion between types must be performed at the orchestration layer before any other utilities can easily leverage it.

## Reproducibility and Integration Exercise

The following components were selected to integrate into pipelines using either modularity Method 1 or 2:

**Perception: Segmentation**

- Point Cloud Library (PCL)[17]
- Unseen Object Clustering (UOC) (Xiang et al. 2021)

**Grasp Planning**

- Contact-GraspNet (CGN) (Sundermeyer et al. 2021)
- Grasp Pose Detection (GPD) (Ten Pas et al. 2017)
- GraspSAM (GSAM) grasp detection (Noh et al. 2025)

It should be noted that while GSAM integrates Grounding DINO[18] for object selection, the image encoder of the Segment Anything Model 2 (SAM 2)[19] for segmentation, and their own grasp detector, the former is not available in the GSAM repository. This selection of components provides a variety of inputs, outputs, camera positions, and modularity methods; see Table 1 for each component's relevant properties. Additionally, each open-source component has varying levels of maturity and popularity in the ecosystem; for example, PCL is widely used to filter point clouds (released in 2011) while GSAM is a fairly new grasp planning solution (released in 2025) with some users. A variety of component characteristics is essential to properly exercise reproducibility and integration that can broadly represent the types of issues that may be encountered, but additional exercises with more components in should be conducted in the future.

### Reproducing Open-Source Components

Of the five open-source components selected for this exercise, we were able to reproduce functionality and performance similar to what is outlined in their respective GitHub repositories and papers for four of them: CGN, GPD, PCL, and UOC. To determine this, we followed the instructions provided in their corresponding documentation, conducting any recommended tests with example data (if provided), and compared the outputs of our reproductions to example qualitative or quantitative data (if provided, either in the repository or original publication). For example, the repository for CGN[3] includes a pre-trained checkpoint and a set of test scenes with example visualizations of planned grasps using that checkpoint. After downloading and installing the pre-trained checkpoint (i.e., our reproduction of CGN), we ran it against this test set for qualitative comparison to the example outputs provided; see Figure 2.

Similarly, the repository for UOC[20] includes example outputs on test scenes used in the original publication (Xiang et al. 2021) and scripts to test its performance on existing datasets like the Tabletop Object Dataset (TOD) (Xie et al. 2020), Open Clutter Indoor Dataset (OCID) (Suchi et al. 2019), and the Object Segmentation Database (OSD)[21], with pre-trained checkpoints for each. We ran our reproduction of the UOC checkpoint trained on TOD and qualitatively compared its outputs to the provided example; for continuity,

[17] https://pointclouds.org/
[18] https://github.com/IDEA-Research/GroundingDINO
[19] https://github.com/facebookresearch/segment-anything
[20] https://github.com/NVlabs/UnseenObjectClustering
[21] https://www.acin.tuwien.ac.at/en/vision-for-robotics/

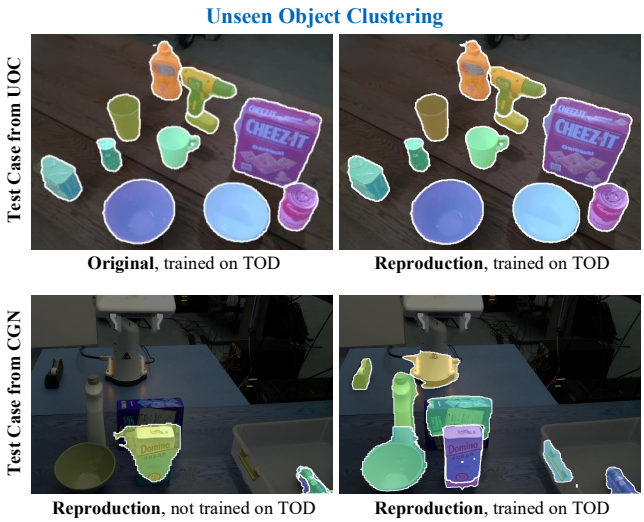



Figure 3: Top: Comparison of the original UOC on an example scene provided in its repository[20] and our reproduction. Bottom: Comparison of our reproduction of UOC on the CGN example scene used in Figure 2 using its provided pre-trained checkpoints trained on TOD and not.

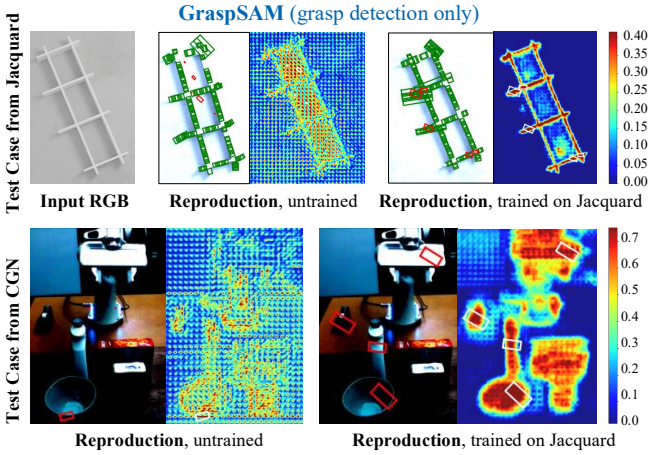



Figure 4: Comparison of our reproduction of GSAM grasp detection checkpoints untrained and trained on the Jacquard dataset[22] on an example image from the Jacquard dataset (top) and on the CGN example scene used in Figure 2 (bottom). Note: the Jacquard dataset includes ground truth 2D grasp rectangles in green for each object and compares the grasps detected by the system (shown as red rectangles) to evaluate their accuracy.

we compared the outputs of this checkpoint to another not trained on TOD using the CGN test scenes (see Figure 3).

The remaining open-source component is GSAM, which was difficult to effectively reproduce in terms of functionality, resulting in performance quite different from the original paper (Noh et al. 2025). The GSAM code retrains its SAM model on the grasp detection datasets – Jacquard Dataset[22] and Grasp-Anything[23] – to obtain checkpoints with image segmentations used for targeting grasp detection. However, the GSAM repository[24] does not provide documentation for grasp detection trained checkpoints – only object detection checkpoints using SAM (EfficientSAM and MobileSAM) – nor does it include the prompt-driven Grounding DINO object detection modules. Using the checkpoints provided in the repository, the algorithm performs poorly even on the default Jacquard dataset. To obtain operational performance, we retrained the model locally on the Jacquard dataset to generate new checkpoints, and integrated additional components including UOC for object segmentation. These modifications enabled the generation of reasonable grasp and manipulation outputs; however, the resulting system does not constitute a direct representation of the original model.

As a test of our reproduction of GSAM, we compared the outputs achieved from using the untrained grasp detection checkpoint in its repository and our checkpoint trained on the Jacquard dataset against the evaluation images in the Jacquard dataset; see Figure 4 (top) for a qualitative comparison. The original GSAM paper also includes cross-dataset quantitative tests of its grasp detection using the Jacquard or Grasp-Anything datasets, presented as a means of comparison to another grasp detection method. Using the checkpoints from the repository resulted in substantially degraded performance, yielding a score of approximately 0.25 on Jacquard. We ran the same script provided by GSAM to compare the results of our reproduction of GSAM grasp detection to that in the original paper. In that paper, GSAM grasp detection trained on Jacquard achieved a score of 0.87 when tested on Jacquard. Our reproduction retrained on Jacquard achieved a score of 0.85 on Jacquard, making for an acceptable reproduction.

For continuity, we ran our untrained and trained GSAM checkpoints on the same test scenes provided by CGN as a qualitative test; see Figure 4 (bottom). While not all of the detected grasps are viable (e.g., those on the bowl), the resulting grasp quality map on the trained checkpoint appears sufficient. Given that GSAM grasp detection is trained on images from a top-down camera view and the CGN test scenes use an oblique view (see Table 1), we did not expect GSAM to perform well. However, when we deploy it in a pipeline, it will use a top-down camera as intended.

## Modularizing Open-Source Components

The underlying libraries of GPD and PCL are compatible with Ubuntu 24.04, so they were able to be wrapped in ROS 2 (Method 1), while CGN, GSAM, and UOC required the subprocess bridge (Method 2). Other than GSAM, open-source ROS 1 wrapped versions of all other components were already available[25,26,27,28] giving us confidence that

[22] https://jacquard.liris.cnrs.fr/
[23] https://airvlab.github.io/grasp-anything/
[24] https://github.com/gist-ailab/GraspSAM
[25] http://wiki.ros.org/pcl_ros
[26] https://github.com/IRVLUTD/UnseenObjectClustering
[27] https://github.com/atenpas/gpd_ros
[28] https://github.com/IRVLUTD/contact_graspnet

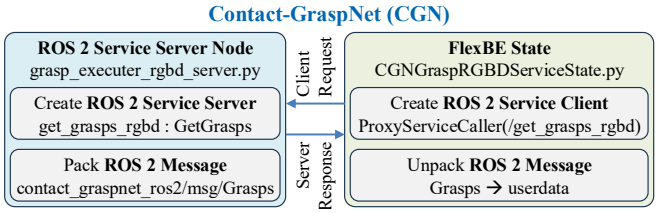



Figure 5: Example connection of a ROS 2 wrapped open-source component (CGN) to a FlexBE state.

developing ROS 2 wrappers would be achievable. There were also several examples of these components being used in a pipeline for picking in clutter (e.g., the repositories for UOC and CGN are from Khargonkar et al. (2024)), providing further confidence they would be performant. We did not utilize modularity Methods 3 or 4 as part of this exercise.

Following our methodology, standard ROS 2 message types were used as often as possible. For example, in PCL, the service definition file to perform Euclidean clustering requests `sensor_msgs` and `geometry_msgs`, responding without additional processing required. Whereas in GPD, the service definition file to detect grasps requests `gpd_ros`, a non-standard message type, incurring additional overhead with the inclusion of a custom library in order for the orchestration layer to understand how to construct the message to pass information to or from it. This was a relic from the ROS 1 wrapped version of GPD; if a ROS 2 wrapper were developed from scratch, standard message types should be used.

To confirm that modularization was successful, we validated integration at the ROS 2 interface level. Each wrapped component was required to expose properly defined topics, services, or actions. Successful modularization was determined by verifying that downstream components could utilize the published outputs without modification and perform their intended function (e.g., a grasp planner receiving the expected point cloud, or a motion planner receiving a valid pose message to plan and execute a trajectory).

Validation was confirmed first by ensuring that individual nodes could correctly publish messages, subscribe to topics, and advertise a service or action server as expected. Visualization tools were used to confirm correctness of outputs, ensuring that the component communicated successfully and produced operationally valid results. For testing the functionality of components made modular using Method 2, we simply check if a host terminal from the ROS 2 server with the subprocess provides the same output as a terminal directly running commands inside the Docker container.

## Integrating Modular Components into a Pipeline

Of the pipeline orchestration methods available, we considered options for state machines and behavior trees.With prior experience using these solutions in ROS 1, proving their capability at supporting the level of modularity and execution control desired, we elected to use FlexBE.

Within our pipelines, FlexBE states are constructed so each requests a service or makes a call to an action server, performing a single task to allow for a high degree of modularity when building a behavior (e.g., see Figure 5 for an example connection of a ROS 2 wrapped component to a FlexBE state). The corresponding FlexBE service state imports the same service type as the ROS 2 service server node, constructs a request from its input userdata, then calls the ROS 2 service through FlexBE's *ProxyServiceCaller*, and copies the returned fields into output userdata for downstream states. This structure decouples the algorithm implementation from the behavior logic, allowing the same FlexBE behavior to reuse or replace wrapped components as long as their service interface remains consistent.

Using FlexBE, we developed three different pipelines that vary perception and grasp planning components while keeping object selection, motion planning, and control components static: Pipeline A (PCL + GPD), B (UOC + CGN), and C (UOC + GSAM); see Table 2. Due to the issues with reproducing GSAM, Pipeline C utilizes UOC for perception as we had already successfully used it in Pipeline B to produce segmentations. This is an example of how modular components enable further interoperability so long as the required input and output criteria between components is met.

To enable seamlessly integration of the open-source components into perception-to-action pipelines, we additionally developed a set of utility modules to standardize data representations and coordinate transformations across models. Such utilities include extensions for grasp planners with 2D grasp rectangle outputs that incorporate externally provided masks for object selection, filter feasible grasp rectangles based on gripper constraints, and convert planar 2D grasp rectangles into 6-DoF grasp poses. Other utilities include generic mask-based target object selection modules following object segmentation (e.g., from UOC), as well as transformation modules that convert 6-DoF grasp poses (e.g., from CGN or transformed from GSAM outputs) from the camera frame into the robot's motion planning frame. The resulting poses are formatted for direct consumption by OMPL-based motion planners, enabling end-to-end trajectory generation and execution.

Unlike the tests conducted when reproducing the open-source components, we were not able to compare the functionality of these pipelines with previously published results. Instead, a basic check of the inputs and outputs of the pipeline components was conducted, ensuring the requirements for each individual component were met. More specifically, so long as the start of the pipeline receives the expected sensor data to its perception component, and then – after executing the rest of the pipeline – results in a 6-DoF grasp pose in the format required by the motion planning component, then the pipeline's functionality is confirmed.

| Pipeline | Perception | Object Selection | Grasp Planning | Motion Planning | Control |
|---|---|---|---|---|---|
| **A** | PCL | Near-to-Far | GPD | OMPL | MoveIt |
| **B** | UOC | Manual | CGN | OMPL | MoveIt |
| **C** | UOC | Manual | GSAM* | OMPL | MoveIt |

Table 2: Pipelines developed using modular components. * Only the grasp detection functionality of GSAM was used.

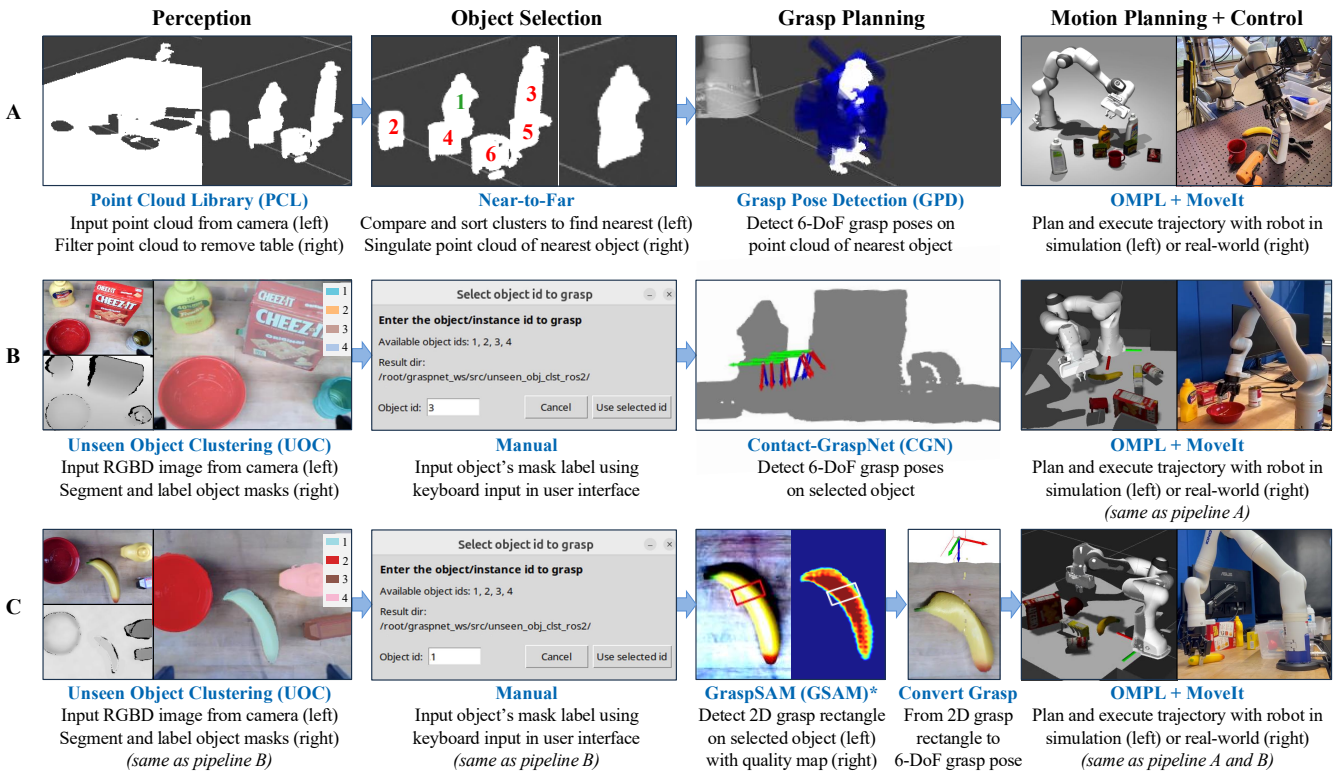



Figure 6: Deploying the pipelines in simulation and real-world. * Only the grasp detection functionality of GSAM was used.

## Deploying Pipelines

We attempted to deploy the pipelines first in simulation and then using physical robots. For both cases, we opted to diversify deployments by using two simulators – (1) Gazebo[29], a basic simulator commonly utilized with ROS; (2) MuJoCo[30], a more advanced simulator often used for robot manipulation – and two physical robot deployments – (1) Universal Robots UR5e arm with Robotiq 2F-85 end-effector and Intel RealSense D435i sensor on its wrist; (2) Kinova Gen3 7-DoF arm with integrated Robotiq 2F-85 end-effector and Intel RealSense D410 sensor on its wrist; see Figure 6. At this stage, the deployments were initial tests of pipeline functionality, not full benchmarking exercises.

Pipeline A successfully used point cloud data input from the robot's camera with PCL to filter it, a custom near-to-far function was run to select the object in the point cloud closest to the robot's camera, GPD planned grasps for this object, the resulting 6-DoF grasp pose is sent to OMPL to plan a trajectory, which is then executed by MoveIt. Pipeline B successfully took an RGBD image from the robot's camera to UOC to segment and label it, an object segment is then manually selected by the user through a custom pop-up interface, CGN plans grasps for the object, and the resulting 6-DoF grasp pose is sent to OMPL, following the same Motion Planning + Control as pipeline A. Pipeline C followed the same Perception, Object Selection, and Motion Planning + Control stages as pipeline B, but with the GSAM grasp detector planning grasps on the object as a 2D grasp rectangle, which then were converted to a 6-DoF grasp pose. All were functional, so the deployments are considered successful.

## Discussion

This exercise illuminated several issues that concur with general sentiments shared throughout the community regarding reproducing and integrating open-source in robotics, reinforcing previously identified challenges in other domains (e.g., service robots (García et al. 2020)) and broader software engineering principles (e.g., (Myers, Sandler, and Badgett 2011)). While attempting to reproduce the selected open-source components, the lack of objective functionality and performance tests limited our understanding as to whether or not reproduction was successful. Due to the materials available in each repository, the tests conducted were qualitative rather than quantitative, the latter of which would have been more definitive tests of reproducibility. The original UOC paper (Xiang et al. 2021) does include quantitative tests of its performance on the OCID and OSD datasets in terms of precision, recall, and F-measure – presented as a means of comparison to another existing segmentation method – but lacked instructions on how to conduct these tests and achieve these results. If specified, this could serve as a reasonable quantitative test of reproducibility.

Many open-source components in this domain are trained on one or more datasets prior to implementation, which can require significant compute resources. In the original GSAM

[29]https://github.com/gazebosim/gz-sim

[30]https://github.com/google-deepmind/mujoco

paper (Noh et al. 2025), the highest reported performance was achieved when it was trained on the Grasp-Anything dataset. However, given the trained grasp detection checkpoints were not available in the repository, reproducing these results would require retraining locally, the compute sources of which were not available to us during the exercise. Consequently, retraining was conducted only on the Jacquard dataset, which did not perform as well. If a pretrained checkpoint had been provided (e.g., like CGN), we would have been able to more effectively reproduce GSAM.

When integrating the modules into a pipeline, the GPD and CGN grasp planners were relatively straightforward as both directly output 6-DoF grasp poses, allowing for simple passing of this information downstream to motion planning for pipelines A and B (respectively). In contrast, integrating GSAM for pipeline C required the conversion of its 2D grasp rectangle output into executable 6-DoF grasp poses before the resulting grasp pose was able to be sent downstream to motion planning; we had to develop this utility ourselves. The GSAM repository also lacked the required assets for utilizing SAM 2 and Grounding DINO for its purported language- or prompt-guided grasping. As a result, we used the assets we previously implemented in pipeline B to perform object segmentation using UOC and the text input interface to select a desired object/mask for grasp planning. While not planned, this conundrum ultimately resulted in us demonstrating the benefits of modularity, enabling the use of the same module (UOC) in two different pipelines.

## Recommendations for Guidelines

Based on our adventures attempting to reproduce and integrate open-source components into pipelines, we recommend the following to increase increasing accessibility, reproducibility, and integration using modular components:

**Provide functionality tests**: Repositories should include materials such as example data and metrics for comparison so users can test if their reproduction functions properly. While qualitative tests are useful, quantitative tests (e.g., sample scenes with success rates) are ideal for objective evaluation and benchmarking performance.

**Provide pretrained checkpoints**: If the performance of a component relies on proper training, then repositories should include pretrained checkpoints that match what is used in its corresponding paper. Given the computing assets typically needed for training, users should not be required to retrain the component to properly reproduce it.

**Use ROS 2 wrappers for modularity**: Take advantage of the ROS 2 ecosystem to make components modular, and if a component is not compatible with your version of ROS 2, use the subprocess bridge method. The repository for the component should be constructed as an individual ROS package so that it can be generally used by others, wrapping the target library and limiting additional non-standard dependencies when possible.

**Use existing ROS message types**: To increase applicability of modular components across multiple use cases, store input and output data using existing ROS 2 message types rather than custom ones. The use of a custom message type may induce requirements at the orchestration layer, thereby limiting the potential reuse of the module in other pipelines without additional custom tailoring.

**Include required utilities for integration**: Some components output formats that are not directly usable by other pipelines (e.g., 2D grasp rectangles must be converted to 6-DoF grasp poses). Repositories should include all utilities required for pipeline integration to limit the amount of custom tooling that must be developed during integration.

**Reproduce before release**: Before making a repository public for others to access, attempts to reproduce it should be made, ideally by a third party. Component-level reproducibility can be assessed by following the functionality tests provided in the repository. Pipeline-level assessments should also be made by attempting to integrate a component to function with others that match its intended use cases and requirements, similar to the exercises conducted in this paper. Assessment results should be reviewed by the developers, addressing identified issues as needed.

## Conclusion and Future Work

This paper reviews a methodology and exercises conducted for reproducing and integrating modular open-source software components into pipelines for picking in clutter. Based on our experiences, several recommendations for guidelines are made for improving the current open-source ecosystem in robot grasping and manipulation. Our next steps are to conduct benchmarking of the pipelines to demonstrate and compare their performance, following the ManipulationNet Grasping in Clutter protocol (Chen et al. 2026), contributing at least one score per pipeline to their online leaderboard[31].

The resulting recommendations will be disseminated throughout the community for feedback, revision, and eventual adoption. Additional mechanisms will be needed for further propagation, though, such as developing "badges" to indicate compliance with the guidelines (e.g., Shields.io[32] for GitHub repos, or those used by ACM for research artifacts[33]) and methods to more easily share the results of reproducibility and integration exercises. The latter is currently done either through posting issues on a GitHub page or publishing using formats like the IEEE RAS R-article (Bonsignorio 2017), although this typically involves reproduction of an entire experiment rather than an individual component. The results of these exercises could be used to rate the ease of reproducibility, integration, deployment, etc., with those ratings available for users to consider while reviewing potential components to leverage. Ideally, by following similar practices to reproduce and integrate more open-source components into more pipelines, thorough assessments of component benchmarking can be conducted across a variety of pipeline combinations.

[31] https://manipulation-net.org/leaderboards.html

[32] https://github.com/badges/shields

[33] https://www.acm.org/publications/policies/artifact-review-and-badging-current

## Acknowledgments

This work and the COMPARE Ecosystem is supported in part by the National Science Foundation (2346069). Thank you to our collaborators that comprise the COMPARE project team from Rutgers University, University of South Florida, Worcester Polytechnic Institute, and Yale University, as well as the broader COMPARE community for their continued engagement towards improving open-source and benchmarking for robot manipulation. As a community-based effort, COMPARE relies on participation from robot manipulation researchers, open-source developers, and industry users. All interested parties are encouraged to provide feedback by visiting robot-manipulation.org, join the Slack and/or Google Group, and reach out to the authors.

## References

Aghajani, E.; Nagy, C.; Vega-Márquez, O. L.; Linares-Vásquez, M.; Moreno, L.; Bavota, G.; and Lanza, M. 2019. Software documentation issues unveiled. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, 1199–1210. IEEE.

Bonsignorio, F. 2017. A new kind of article for reproducible research in intelligent robotics [from the field]. *IEEE Robotics & Automation Magazine*, 24(3): 178–182.

Bottarel, F.; Altobelli, A.; Pattacini, U.; and Natale, L. 2023. GRASPA-fying the Panda: Easily Deployable, Fully Reproducible Benchmarking of Grasp Planning Algorithms. *IEEE Robotics & Automation Magazine*.

Canelas, P.; Schmerl, B.; Fonseca, A.; and Timperley, C. S. 2024. Understanding misconfigurations in ROS: An empirical study and current approaches. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 1161–1173.

Cervera, E. 2018. Try to start it! The challenge of reusing code in robotics research. *IEEE Robotics and Automation Letters*, 4(1): 49–56.

Cervera, E. 2023. Run to the source: The effective reproducibility of robotics code repositories. *IEEE Robotics & Automation Magazine*, 31(2): 125–134.

Cervera, E. 2025. One-Click Reproducibility of Robotics Software: A Graphical Workflow Solution. *IEEE Robotics & Automation Magazine*.

Chen, Y.; Kimble, K.; Adelson, E. H.; Asfour, T.; Chanrungmaneekul, P.; Chitta, S.; Chitambar, Y.; Chen, Z.; Goldberg, K.; Kragic, D.; et al. 2026. ManipulationNet: An infrastructure for benchmarking real-world robot manipulation with physical skill challenges and embodied multimodal reasoning. *arXiv preprint arXiv:2603.04363*.

D'Avella, S.; Bianchi, M.; Sundaram, A. M.; Avizzano, C. A.; Roa, M. A.; and Tripicchio, P. 2023. The cluttered environment picking benchmark (CEPB) for advanced warehouse automation: Evaluating the perception, planning, control, and grasping of manipulation systems. *IEEE Robotics & Automation Magazine*, 31(4): 45–58.

García, S.; Strüber, D.; Brugali, D.; Berger, T.; and Pelliccione, P. 2020. Robotics software engineering: A perspective from the service robotics domain. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 593–604.

Khargonkar, N.; Allu, S. H.; Lu, Y.; Prabhakaran, B.; and Xiang, Y. 2024. SceneReplica: Benchmarking real-world robot manipulation by creating replicable scenes. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 8258–8264. IEEE.

Kolak, S.; Afzal, A.; Le Goues, C.; Hilton, M.; and Timperley, C. S. 2020. It takes a village to build a robot: An empirical study of the ROS ecosystem. In *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 430–440. IEEE.

Liu, J.; Yap, H. J.; and Khairuddin, A. S. M. 2024. Review on motion planning of robotic manipulator in dynamic environments. *Journal of Sensors*, 2024(1): 5969512.

Lynnerup, N. A.; Nolling, L.; Hasle, R.; and Hallam, J. 2020. A survey on reproducibility by evaluating deep reinforcement learning algorithms on real-world robots. In *Conference on Robot Learning*, 466–489. PMLR.

Myers, G. J.; Sandler, C.; and Badgett, T. 2011. *The art of software testing*. John Wiley & Sons.

Newbury, R.; Gu, M.; Chumbley, L.; Mousavian, A.; Eppner, C.; Leitner, J.; Bohg, J.; Morales, A.; Asfour, T.; Kragic, D.; Fox, D.; and Cosgun, A. 2023. Deep learning approaches to grasp synthesis: A review. *IEEE Transactions on Robotics*, 39(5): 3994–4015.

Noh, S.; Kim, J.; Nam, D.; Back, S.; Kang, R.; and Lee, K. 2025. GraspSAM: When segment anything model meets grasp detection. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 14023–14029. IEEE.

Pinho, G.; Caçula, A. J.; Costa, L.; Wiese, I.; and Araújo, A. A. 2024. Challenges and Solutions of Free and Open Source Software Documentation: A Systematic Mapping Study. *Simpósio Brasileiro de Engenharia de Software (SBES)*, 114–125.

Quigley, M.; Conley, K.; Gerkey, B.; Faust, J.; Foote, T.; Leibs, J.; Berger, E.; Wheeler, R.; and Ng, A. 2009. ROS: An open-source Robot Operating System. In *ICRA Workshop on Open Source Software*, volume 3, 5. Kobe, Japan.

Rameshbabu, B. K.; Balakrishna, S. S.; Flynn, B.; Kapoor, V.; Norton, A.; Yanco, H.; and Calli, B. 2025. A Benchmarking study of vision-based robotic grasping algorithms: a comparative analysis. *IEEE Robotics & Automation Magazine*.

Suchi, M.; Patten, T.; Fischinger, D.; and Vincze, M. 2019. EasyLabel: A semi-automatic pixel-wise object annotation tool for creating robotic RGB-D datasets. In *2019 International Conference on Robotics and Automation (ICRA)*, 6678–6684. IEEE.

Sundermeyer, M.; Mousavian, A.; Triebel, R.; and Fox, D. 2021. Contact-GraspNet: Efficient 6-DOF grasp generation in cluttered scenes. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 13438–13444. IEEE.

Ten Pas, A.; Gualtieri, M.; Saenko, K.; and Platt, R. 2017. Grasp pose detection in point clouds. *The International Journal of Robotics Research*, 36(13-14): 1455–1473.

Tola, D.; and Corke, P. 2023. Understanding URDF: A survey based on user experience. In *2023 IEEE 19th International Conference on Automation Science and Engineering (CASE)*, 1–7. IEEE.

Tola, D.; and Corke, P. 2024. Understanding URDF: A dataset and analysis. *IEEE Robotics and Automation Letters*, 9(5): 4479–4486.

Tong, E.; Balaji, A.; Opipari, A.; Lewis, S.; Zeng, Z.; and Jenkins, O. C. 2025. OVAL-Grasp: Open-Vocabulary Affordance Localization for Task Oriented Grasping. In *19th International Symposium on Experimental Robotics (ISER) 2025*.

Um, S.; Son, Y. G.; Shim, J.; and Choi, H. R. 2026. Toward Reliable Bin-Picking: Collision-Aware Robotic Design and Control Strategy for Heavily Cluttered Environment. *IEEE Robotics and Automation Practice*.

Xiang, Y.; Xie, C.; Mousavian, A.; and Fox, D. 2021. Learning RGB-D feature embeddings for unseen object instance segmentation. In *Conference on Robot Learning*, 461–470. PMLR.

Xie, C.; Xiang, Y.; Mousavian, A.; and Fox, D. 2020. The best of both modes: Separately leveraging RGB and depth for unseen object instance segmentation. In *Conference on Robot Learning*, 1369–1378. PMLR.